

Lab Assignment: Real-Time Object Detection with YOLO

Course: Deep Learning - DVA307

Lab: 3

Tools: Labellmg, Google Colab, Python (Streamlit + Ultralytics)

1. Objective

The goal of this lab is to build a complete, end-to-end Machine Learning pipeline. You will not just train a model; you will create the dataset, annotate it, train a state-of-the-art **YOLO11 Nano** model in the cloud, and deploy it locally on your laptop to create a real-time detection application.

2. Prerequisites

- **Hardware:** Laptop with a working webcam.
- **Software:** Python installed (version 3.8 or higher).
- **Accounts:** Google Account (to access Google Colab).

Part 1: Data Collection & Local Annotation

Task: Create a custom dataset for a single object class (e.g., Coffee Mug, Mouse, Sunglasses, Watch).

Step 1.1: Image Collection

1. Take **100-120 photos** of your chosen object.
2. **Vary the conditions:**
 - **Angles:** Top-down, side view, front view.
 - **Lighting:** Bright light, dim light, shadows.
 - **Background:** Clean desk, cluttered floor, held in hand.
 - **Distance:** Close-up, far away.
3. Create a folder on your Desktop named `my_dataset`.
4. Inside it, create a subfolder named `images` and paste all photos there.

Step 1.2: Install Labellmg

We will use **Labellmg**, a free tool for graphical image annotation.

1. Open your Command Prompt (Windows) or Terminal (Mac/Linux).
2. Run the following command to install it:
`pip install labellmg`

3. Launch the tool by typing:
labellmg

Step 1.3: Annotate Images (Crucial Step)

1. **Set Format to YOLO:** Look at the toolbar on the left. Find the button that says "PascalVOC". **Click it until it changes to "YOLO"**.
 - *Warning:* If you skip this, your model will not train.
2. **Open Directory:** Click "**Open Dir**" and select your my_dataset/images folder.
3. **Set Save Directory:** Click "**Change Save Dir**" and select the **same folder** (my_dataset/images).
4. **Label:**
 - Press **W** on your keyboard to draw a box.
 - Draw a tight box around the object.
 - Type your label name (e.g., mouse) and press OK.
 - Press **D** to move to the next image.
5. Repeat this for all images. You will see .txt files appear next to your images.

Step 1.4: Organize and Zip

YOLO requires a specific folder structure.

1. Inside my_dataset, create this structure:
 - train/images
 - train/labels
 - val/images
 - val/labels
2. **Split your data:**
 - Move **80%** of your image/text pairs into the train folders.
 - Move **20%** of your image/text pairs into the val folders.
3. **Compress:** Zip the entire my_dataset folder into a file named dataset.zip.

Part 2: Cloud Training (Google Colab)

Task: Train the model using a free GPU in the cloud.

Step 2.1: Setup Environment

1. Go to colab.research.google.com.
2. Click **New Notebook**.
3. Go to **Runtime > Change runtime type > Select T4 GPU > Save**.

Step 2.2: Upload Data

1. Click the **Folder Icon** on the left sidebar.
2. Drag and drop your dataset.zip into the file area.

3. Run this code in a cell to unzip it:

```
!unzip -q dataset.zip -d /content/
```

Step 2.3: Configure Training

1. Install the YOLO library:

```
!pip install ultralytics
```

2. Create the dataset configuration file (data.yaml):

```
import yaml
```

```
# UPDATE 'names' with your actual object label
```

```
data_config = {  
    'path': '/content/my_dataset',  
    'train': 'train/images',  
    'val': 'val/images',  
    'nc': 1,  
    'names': ['my_object_name']  
}
```

```
with open('/content/data.yaml', 'w') as f:
```

```
    yaml.dump(data_config, f)
```

Step 2.4: Train Model

Run the training command (this will take 10-20 minutes):

```
from ultralytics import YOLO
```

```
# Load the Nano model (Smallest & Fastest)
```

```
model = YOLO('yolo11n.pt')
```

```
# Train for 50 epochs
```

```
results = model.train(  
    data='/content/data.yaml',  
    epochs=50,  
    imgsz=640,  
    plots=True  
)
```

Step 2.5: Download Result

1. When training finishes, navigate (left sidebar) to: runs > detect > train > weights.
2. Right-click **best.pt** and select **Download**.
3. *Note:* Also find results.png or confusion_matrix.png in the train folder and save a screenshot for your report.

Part 3: Real-Time Laptop Deployment

Task: Deploy your model locally using a Python Virtual Environment.

Step 3.1: Create Project Environment

1. Install Python **version 12** in your system
2. Create a folder on your desktop named yolo_project.
3. Paste your best.pt file inside.
4. Open your Terminal/Command Prompt and navigate to the folder:
cd Desktop/yolo_project
5. **Create a Virtual Environment** (Best Practice):
python -m venv venv
6. **Activate the Environment:**
 - **Windows:** venv\Scripts\activate
 - **Mac/Linux:** source venv/bin/activate
 - *(You should see (venv) appear in your terminal).*
7. **Install Dependencies:**
pip install ultralytics streamlit opencv-python

Step 3.2: Create the Application

Create a file named app.py in the folder and paste this code:

```
import streamlit as st
import cv2
from ultralytics import YOLO

# 1. Load Model
model = YOLO("best.pt")

st.title("🔴 Real-Time Object Detection")
st.write("Ensure your webcam is enabled.")

# 2. UI Elements
frame_placeholder = st.empty()
```

```
stop_button = st.button("Stop Camera")

# 3. Initialize Camera (ID 0 is default webcam)
cap = cv2.VideoCapture(0)

while cap.isOpened() and not stop_button:
    ret, frame = cap.read()
    if not ret:
        st.write("Video capture ended.")
        break

# 4. Inference
results = model(frame)

# 5. Visualize (Plot bounding boxes)
annotated_frame = results[0].plot()

# 6. Display (Convert BGR to RGB for Browser)
frame_rgb = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)
frame_placeholder.image(frame_rgb, channels="RGB")

# Allow loop break with 'q' key
if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()
```

Step 3.3: Run the App

In your terminal (with venv activated), run:

```
streamlit run app.py
```

A browser window will open. Test your model by holding your object in front of the camera.

Part 4: Submission & Evaluation

A. PDF Report Requirements

Submit your training Jupyter notebook and a **single report PDF** containing:

1. **Dataset Evidence:** Screenshot of your folder and Labelling window.
2. **Training Analysis:** Screenshot of the **Loss Graphs** from Colab.
3. **Metrics:** State your final **mAP50** score.
4. **Proof of Function:** Screenshot of your Streamlit app detecting the object.
5. **Reflection:** 2-3 sentences on what difficulties you faced (e.g., lighting, false positives).
6. • **Lab Questions (Answer the following in the lab report):**
 - **Q1 (Data Diversity):** How did the background and lighting variations in your dataset affect the model's performance? Did it fail in specific conditions (e.g., low light)?
 - **Q2 (Training):** Look at your Training Loss vs. Validation Loss graph. Did your model show signs of overfitting (memorizing data) or underfitting (failing to learn)? How can you tell?
 - **Q3 (Architecture):** We used yolo11n (Nano). Why is this model preferred for a real-time laptop application over a larger model like yolo11x (Extra Large)? What is the trade-off?
 - **Q4 (Deployment):** In the app.py code, we run model(frame) inside a while loop. What would happen to the video feed if the model took 1 second to process a single image?
7. **Please use high quality screenshots, proper formatting and sectioning in your report.**

B. Live Demo & Viva (In-Lab)

You will present your working app to the instructor.

Demo Checklist:

- Laptop environment set up and ready.
- App detects the object continuously in real-time.

Instructor will ask a few questions about the lab topic.